

Coverage tells you what a model has not seen. It does not tell you what it cannot do.

Blog topic 19: Demonstration Coverage as a Predictor of Extrapolation Success.

The falsifiable claim: for a model that iterates a shared update at inference, training coverage does not predict extrapolation accuracy unless you condition on the inference iteration budget. Report accuracy at one budget and you will misread a compute shortfall as a generalisation failure.

Coverage is the usual first suspect when a model fails on instances harder than anything it trained on, and the reasoning is sound: a seven-step problem is out of distribution if training held none. But in recurrent models difficulty passes through two independent gates, and coverage is only one of them.

A toy where the two gates come apart

I trained a small shared-weight recurrent network on directed graph reachability: given a graph, a source and a target, decide whether a path exists. One update block is applied R times at inference. The parameter count is 11,169 at every R , so R is a dial rather than a different model. Training saw shortest-path distances of one to four only. This is a toy, not a language model. Its value is that the two gates are separable in it.

Five seeds, frozen checkpoints, identical inputs. Only R changes.

Inference budget R	Accuracy on distances ≥ 5 , never trained on
4	0.500
6	0.667
8	0.833
10	0.988

Coverage is constant down that column. A coverage-based predictor evaluated at $R = 4$ reports total extrapolation failure. The same weights and the same training distribution, evaluated at $R = 10$, report near-perfect extrapolation.

The mechanism is one sentence: information travels at most one edge per iteration, so a distance- d instance is undecidable from the target's state until $R \geq d$. Below that line the question is not hard, it is unasked. The same structure appears at scale. Geiping et al. report accuracy rising with test-time recurrence on a 3.5B-parameter model with no weight changes (arXiv:2502.05171, 2025), and Saunshi et al. show looped transformers gaining effective reasoning depth from parameter reuse rather than parameter count (ICLR 2025).

The failure case that breaks the obvious fix

The obvious fix is to detect insufficient budget from the model's own uncertainty. It does not work here. On a distance-7 graph the shipped checkpoint returns 0.000014 for "reachable" at $R = 5$, then 0.999 at $R = 7$. Below the threshold it is confidently wrong, not undecided. Aggregate accuracy near 0.5 at those depths is an average over confident answers that split evenly, not evidence that any single prediction is calibrated. So confidence cannot separate a budget shortfall from a coverage failure, and a coverage predictor built on confidence scores is reading a signal that is not there.

Budget is necessary, not sufficient

Coverage does bind eventually. At distance 10 with $R = 10$ the same checkpoint fails, returning 0.000129 where 0.999 was available at distance 8. Coverage binds, but only after the budget does, so neither number is informative alone.

Where BDH fits

BDH-CQ separates these two axes architecturally rather than by convention. Its contextual state advances once per demonstration, absorbing the task's coverage. A distinct query-time recurrence then iterates with that state held fixed and no new input, and its budget is chosen at inference with no parameter updates (BDH-CQ technical report, arXiv:2608.09888, 2026). Its published operating points move pass@2 on the public ARC-AGI-1 set from 21% to 29.5% across LOW, MEDIUM and HIGH latent effort while coverage is unchanged: the same shape as the table above, at a scale I cannot reproduce. The underlying architecture is Dragon Hatchling, a Post-Transformer family in which attention is reformulated as synaptic memory that updates as the model reads (arXiv:2509.26507, 2025). Those are quoted results, not reproductions.

How to prove me wrong

Retrain the same model with coverage extended to distance 10, then measure distance-7 cases at $R = 4$. If accuracy rises above chance, coverage predicts extrapolation independently of budget and this claim is false. My prediction, stated before the experiment, is that it will not move, because no amount of coverage puts information into a state that information has not reached. That is a judgment, not a result.

Sources

- Geiping, McLeish, Jain et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171, 2025.
- Saunshi, Dikkala, Li, Kumar, Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. ICLR 2025.
- Kosowski, Uznański, Chorowski, Stamirowska, Bartoszkiewicz. The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507, 2025.
- Engdahl, Kosowski, Chorowski et al. BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv:2608.09888, 2026.

Every number attributed to the toy model is reproducible with `python reproduce.py` in the project repository.